

Image Creation Function	What it does	Quick Example
cv2.imread(filename, flags=cv2.IMREAD_COLOR)	Load an image from disk. flags can be IMREAD_GRAYSCALE, IMREAD_COLOR, or IMREAD_UNCHANGED.	img = cv2.imread('cat.jpg')
cv2.imwrite(filename, img, params=None)	Write an image to disk. Optional params control compression (cv2.IMWRITE_JPEG_QUALITY, etc.).	cv2.imwrite('out.png', img)
cv2.imreadmulti(filename, flags=cv2.IMREAD_COLOR)	Read multi-page images (e.g. TIFF stacks). Returns a list of frames.	frames, _ = cv2.imreadmulti('stack.tiff')
cv2.imdecode(buf, flags)	Decode an image from a memory buffer (useful when you have raw bytes).	img = cv2.imdecode(np.frombuffer(raw_bytes, np.uint8), cv2.IMREAD_COLOR)
cv2.imencode(ext, img, params=None)	Encode an image into memory (e.g. PNG or JPEG). Returns a buffer that can be sent over network.	_, buf = cv2.imencode('.jpg', img)
Basic Image Operations		
cv2.cvtColor(src, code)	Convert image from one color space to another (COLOR_BGR2GRAY, COLOR_BGR2HSV, etc.).	gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.resize(src, dsize, fx=0, fy=0, interpolation=cv2.INTER_LINEAR)	Resize image. dsize is (width, height). fx, fy are scaling factors.	small = cv2.resize(img, (200, 200))
cv2.flip(src, flipCode)	Flip image: 0 vertical, 1 horizontal, -1 both.	hflip = cv2.flip(img, 1)
cv2.copyMakeBorder(src, top, bottom, left, right, borderType)	Add borders (BORDER_CONSTANT, BORDER_REFLECT, etc.).	padded = cv2.copyMakeBorder(img, 10,10,10,10, cv2.BORDER_CONSTANT)
cv2.blur(src, ksize)	Simple box filter.	blurred = cv2.blur(img, (5,5))
cv2.GaussianBlur(src, ksize, sigmaX, sigmaY=0)	Gaussian smoothing.	smooth = cv2.GaussianBlur(img, (5,5), 0)
cv2.medianBlur(src, ksize)	Median filter (good for salt-and-pepper noise).	median = cv2.medianBlur(img, 5)
cv2.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C)	Adaptive binarization (e.g. ADAPTIVE_THRESH_MEAN_C).	thresh = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)
cv2.threshold(src, thresh, maxval, type)	Simple thresholding (THRESH_BINARY, THRESH_OTSU, etc.).	_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
cv2.equalizeHist(src)	Histogram equalization (for grayscale).	equalized = cv2.equalizeHist(gray)
cv2.normalize(src, dst=None, alpha=0, beta=255, norm_type=cv2.NORM_MINMAX)	Scale pixel values to a specified range.	norm = cv2.normalize(img, None, 0, 255, cv2.NORM_MINMAX)
cv2.bitwise_and(src1, src2, mask=None)	Element-wise logical AND.	anded = cv2.bitwise_and(img1, img2)
cv2.bitwise_or(src1, src2, mask=None)	Element-wise logical OR.	ored = cv2.bitwise_or(img1, img2)
cv2.bitwise_not(src, mask=None)	Logical NOT.	inverted = cv2.bitwise_not(img)
cv2.add(src1, src2, dst=None, mask=None, dtype=None)	Add two images (with saturation).	added = cv2.add(img1, img2)
cv2.subtract(src1, src2, dst=None, mask=None, dtype=None)	Subtract two images.	sub = cv2.subtract(img1, img2)
cv2.multiply(src1, src2, dst=None, scale=1, dtype=None)	Multiply two images.	mul = cv2.multiply(img1, img2, scale=0.5)
Geometric Transformations		
cv2.getRotationMatrix2D(center, angle, scale)	Compute 2-D rotation matrix.	M = cv2.getRotationMatrix2D((w/2,h/2), 45, 1)
cv2.warpAffine(src, M, dsize, flags=cv2.INTER_LINEAR)	Apply affine transform (rotation, scaling, shearing).	rotated = cv2.warpAffine(img, M, (w,h))
cv2.getPerspectiveTransform(srcPoints, dstPoints)	Compute perspective (homography) matrix.	H = cv2.getPerspectiveTransform(src, dst)
cv2.warpPerspective(src, H, dsize, flags=cv2.INTER_LINEAR)	Apply perspective transform.	warped = cv2.warpPerspective(img, H, (w,h))
cv2.estimateAffinePartial2D(srcPoints, dstPoints)	Estimate affine transform (3x2) from point correspondences.	M, inliers = cv2.estimateAffinePartial2D(src, dst)
cv2.estimateAffine2D(srcPoints, dstPoints)	Estimate full affine transform (includes scaling).	M, inliers = cv2.estimateAffine2D(src, dst)
cv2.findHomography(srcPoints, dstPoints, method=cv2.RANSAC)	Estimate homography matrix using RANSAC.	H, mask = cv2.findHomography(src, dst)
cv2.getAffineTransform(srcPoints, dstPoints)	Compute affine transform from 3 point pairs.	M = cv2.getAffineTransform(np.float32(src), np.float32(dst))
Drawing - Annotation		
cv2.line(img, pt1, pt2, color, thickness=1, lineType=cv2.LINE_8)	Draw a line.	cv2.line(img, (10,10), (100,50), (0,255,0), 2)
cv2.rectangle(img, pt1, pt2, color, thickness=1, lineType=cv2.LINE_8)	Draw a rectangle.	cv2.rectangle(img, (50,50), (200,200), (255,0,0), 3)
cv2.circle(img, center, radius, color, thickness=-1)	Draw a circle.	cv2.circle(img, (120,80), 30, (0,0,255), -1)
cv2.ellipse(img, center, axes, angle, startAngle, endAngle, color, thickness)	Draw an ellipse.	cv2.ellipse(img, (100,100), (50,30), 45, 0, 180, (255,255,0), -1)
cv2.polylines(img, pts, isClosed, color, thickness=1)	Draw a polyline (open or closed).	cv2.polylines(img, [pts], True, (0,255,255), 2)
cv2.putText(img, text, org, fontFace, fontScale, color, thickness)	Write text.	cv2.putText(img, 'Hello', (10,30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255,255,255), 2)
cv2.arrowedLine(img, pt1, pt2, color, thickness, tipLength)	Draw an arrowed line.	cv2.arrowedLine(img, (10,10), (200,50), (0,255,0), 2)
Color Space Conversions		
cv2.cvtColor(src, cv2.COLOR_BGR2XYZ)	Convert to CIE XYZ.	xyz = cv2.cvtColor(bgr, cv2.COLOR_BGR2XYZ)
cv2.cvtColor(src, cv2.COLOR_BGR2LAB)	Convert to CIE LAB.	lab = cv2.cvtColor(bgr, cv2.COLOR_BGR2LAB)
cv2.cvtColor(src, cv2.COLOR_BGR2Luv)	Convert to CIE Luv.	luv = cv2.cvtColor(bgr, cv2.COLOR_BGR2Luv)
cv2.cvtColor(src, cv2.COLOR_BGR2YCrCb)	Convert to YCrCb.	ycrcb = cv2.cvtColor(bgr, cv2.COLOR_BGR2YCrCb)
cv2.cvtColor(src, cv2.COLOR_BGR2HSV_FULL)	Full range HSV (0-255).	hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV_FULL)
cv2.cvtColor(src, cv2.COLOR_GRAY2BGR)	Expand gray to 3-channel BGR.	color = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
Image Filtering - Edge Detection		
cv2.Sobel(src, ddepth, dx, dy, ksize=3)	Compute first-order Sobel derivative.	sobelx = cv2.Sobel(gray, cv2.CV_64F, 1, 0)
cv2.Laplacian(src, ddepth, ksize=1)	Laplacian (second-order) derivative.	lap = cv2.Laplacian(gray, cv2.CV_64F)
cv2.Canny(image, threshold1, threshold2, apertureSize=3, L2gradient=False)	Canny edge detector.	edges = cv2.Canny(gray, 100, 200)
cv2.GaussianBlur(src, ksize, sigmaX)	Gaussian blur (already listed).	blur = cv2.GaussianBlur(img, (5,5), 0)
cv2.blur(src, ksize)	Box blur.	box = cv2.blur(img, (5,5))
cv2.bilateralFilter(src, d, sigmaColor, sigmaSpace)	Bilateral filter (edge-preserving smoothing).	bilat = cv2.bilateralFilter(img, 9, 75, 75)
cv2.morphologyEx(src, op, kernel, iterations=1)	Morphological operations (MORPH_OPEN, MORPH_CLOSE, etc.).	opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
cv2.erode(src, kernel, iterations=1)	Erosion.	eroded = cv2.erode(binary, kernel)
cv2.dilate(src, kernel, iterations=1)	Dilation.	dilated = cv2.dilate(binary, kernel)
Contours - Shape Analysis		
cv2.findContours(image, mode, method)	Find contours in a binary image.	contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
cv2.drawContours(image, contours, contourIdx, color, thickness=-1)	Draw contours.	cv2.drawContours(img, contours, -1, (0,255,0), 2)
cv2.boundingRect(contour)	Get bounding rectangle of contour.	x,y,w,h = cv2.boundingRect(cnt)
cv2.minAreaRect(contour)	Minimum area rotated rectangle.	rect = cv2.minAreaRect(cnt)
cv2.fitEllipse(contour)	Fit ellipse to contour.	ellipse = cv2.fitEllipse(cnt)
cv2.arcLength(contour, closed)	Compute perimeter.	length = cv2.arcLength(cnt, True)
cv2.contourArea(contour)	Compute area.	area = cv2.contourArea(cnt)
cv2.approxPolyDP(curve, epsilon, closed)	Polygonal approximation.	approx = cv2.approxPolyDP(cnt, 0.01*cv2.arcLength(cnt, True), True)
ML-DL Commands		
cv2.ml.SVM_create()	Create a Support Vector Machine.	svm = cv2.ml.SVM_create(); svm.train(samples, cv2.ML_DATA_SAMPLE_COLS, labels)
cv2.ml.KNearest_create()	Create k-nearest neighbours classifier.	knn = cv2.ml.KNearest_create(); knn.train(samples, cv2.ML_DATA_SAMPLE_COLS, labels)
cv2.ml.ANN_MLP_create()	Create a multi-layer perceptron.	mlp = cv2.ml.ANN_MLP_create(); mlp.train(samples, cv2.ML_DATA_SAMPLE_COLS, labels)
cv2.dnn.readNetFromCaffe(prototxt, model)	Load a Caffe network.	net = cv2.dnn.readNetFromCaffe('deploy.prototxt', 'model.caffemodel')
cv2.dnn.readNetFromTensorflow(frozenGraph)	Load a TensorFlow frozen graph.	net = cv2.dnn.readNetFromTensorflow('frozen_inference_graph.pb')
cv2.dnn.readNetFromONNX(onnxModel)	Load an ONNX model.	net = cv2.dnn.readNetFromONNX('model.onnx')
net.setInput(blob)	Set input blob for the network.	blob = cv2.dnn.blobFromImage(img, 1/255, (224,224)); net.setInput(blob)
Video Capture & Processing		
cv2.VideoCapture(index)	Open camera or video file.	cap = cv2.VideoCapture(0)
cap.read()	Read next frame.	ret, frame = cap.read()
cv2.VideoWriter(filename, fourcc, fps, frameSize)	Write video to disk.	out = cv2.VideoWriter('out.mp4', cv2.VideoWriter_fourcc('mp4v'), 30, (w,h))
cap.set(propId, value)	Set camera property (CAP_PROP_FRAME_WIDTH, etc.).	cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
cv2.getTickCount() / cv2.getTickFrequency()	High-resolution timer.	t0 = cv2.getTickCount(); ...; elapsed = (cv2.getTickCount()-t0)/cv2.getTickFrequency()

--	--	--